



BABU BANARASI DAS UNIVERSITY

2026–2027

SCHOOL OF COMPUTER APPLICATIONS

NoSQL Project On

Student Management System Using MongoDB

Submitted to	Submitted by
Mr. Harendra Singh Rajpoot	Diwakar Mishra
Department: BCA (DS & AI)	Roll No. 33
Class: BCA (DS & AI) 23	University: Babu Banarasi Das University
Date: 25 September 2026	Univ Roll No.: 1250258198

PROJECT REPORT

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher **Mr. Harendra Singh Rajpoot** for giving me the opportunity to work on this NoSQL project titled “**Student Management System Using MongoDB**”.

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update, sort and delete data. I also learned how different query operators such as greater than, less than, equal to, AND, and OR are used in MongoDB.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

TABLE OF CONTENTS

1. Introduction
2. Objectives
3. Project Overview and Database Structure
4. Student Dataset
5. MongoDB Implementation and Operations
6. MongoDB Operators Used
7. Conclusion
8. Appendix: Complete Student Dataset

Note: Following the supplied reference document, the implementation section uses execution screenshots rather than reproducing MongoDB queries as typed code. The screenshots show the actual MongoDB command/output style used in the project.

1. INTRODUCTION

MongoDB is a popular NoSQL database management system that stores data in the form of documents. Unlike traditional relational databases, MongoDB does not require data to be stored in fixed rows and columns.

In this project, a **Student Management System** has been created using MongoDB. The database contains information about **50 students**, including roll number, name, age, marks and city.

Different MongoDB operations have been performed on the student data, such as inserting, displaying, updating, deleting, sorting, counting and searching records. Comparison operators such as **\$eq**, **\$gt**, **\$lt**, **\$gte** and **\$lte** have also been demonstrated.

2. OBJECTIVES

- To understand the concept of NoSQL databases.
- To learn the basic features of MongoDB.
- To create a database and collection.
- To insert 50 student records.
- To perform CRUD operations.
- To use comparison operators.
- To perform sorting and filtering.
- To understand AND and OR operations.

3. PROJECT OVERVIEW AND DATABASE STRUCTURE

Project title: Student Management System Using MongoDB

Database: Students

Collection: students

Number of initial student documents: 50

Document fields: roll, name, age, marks, city

The project demonstrates a document-oriented approach in which each student is represented as a MongoDB document. The collection is then queried using retrieval, comparison, logical, sorting, limiting, updating, deleting, searching and counting operations.

Field	Purpose	Example
roll	Unique student roll number	1
name	Student name	Anamika
age	Student age	19
marks	Student marks	88
city	Student city	Lucknow

4. STUDENT DATASET

The reference project defines 50 student records. The dataset contains a mixture of ages, marks and cities and is used for all demonstrations in the implementation section.

Roll	Name	Age	Marks	City
1	Anamika	19	88	Lucknow
2	Uday	18	76	Jaunpur
3	Vaishnavi	19	92	Kanpur
4	Annu	18	69	Ayodhya
5	Vansh	19	81	Varanasi
6	Amrita	20	85	Lucknow
7	Naveen	19	73	Prayagraj
8	Pratik	19	90	Lucknow
9	Sangita	20	78	Gorakhpur
10	Vivek	18	65	Kanpur
11	Tarkeshwar	20	71	Jaunpur
12	Shreya	19	95	Lucknow
13	Akshita	18	84	Ayodhya
14	Shruti	19	87	Prayagraj
15	Kartik	20	74	Varanasi
16	Sejal	18	89	Lucknow
17	Astha	19	82	Kanpur
18	Jai	18	58	Gonda
19	Harshita	20	91	Lucknow
20	Omkar	19	63	Bareilly
21	Ayush	18	77	Lucknow
22	Saumya	19	86	Jaunpur
23	Vidya	20	72	Kanpur
24	Adarsh	19	80	Ayodhya
25	Anmol	18	67	Prayagraj
26	Anushka	20	94	Lucknow
27	Ananya	19	88	Varanasi
28	Bhaskar	18	61	Gorakhpur
29	Rohan	19	75	Kanpur
30	Nishant	20	79	Lucknow
31	Vineet	18	68	Jaunpur
32	Anika	19	90	Lucknow
33	Jai	20	55	Ayodhya
34	Lakshya	18	83	Kanpur
35	Krishna	19	96	Prayagraj
36	Rumana	20	70	Lucknow
37	Priyanshu	18	81	Varanasi
38	Aditya	19	66	Gorakhpur
39	Dhruv	20	93	Lucknow
40	Nivedita	19	85	Kanpur

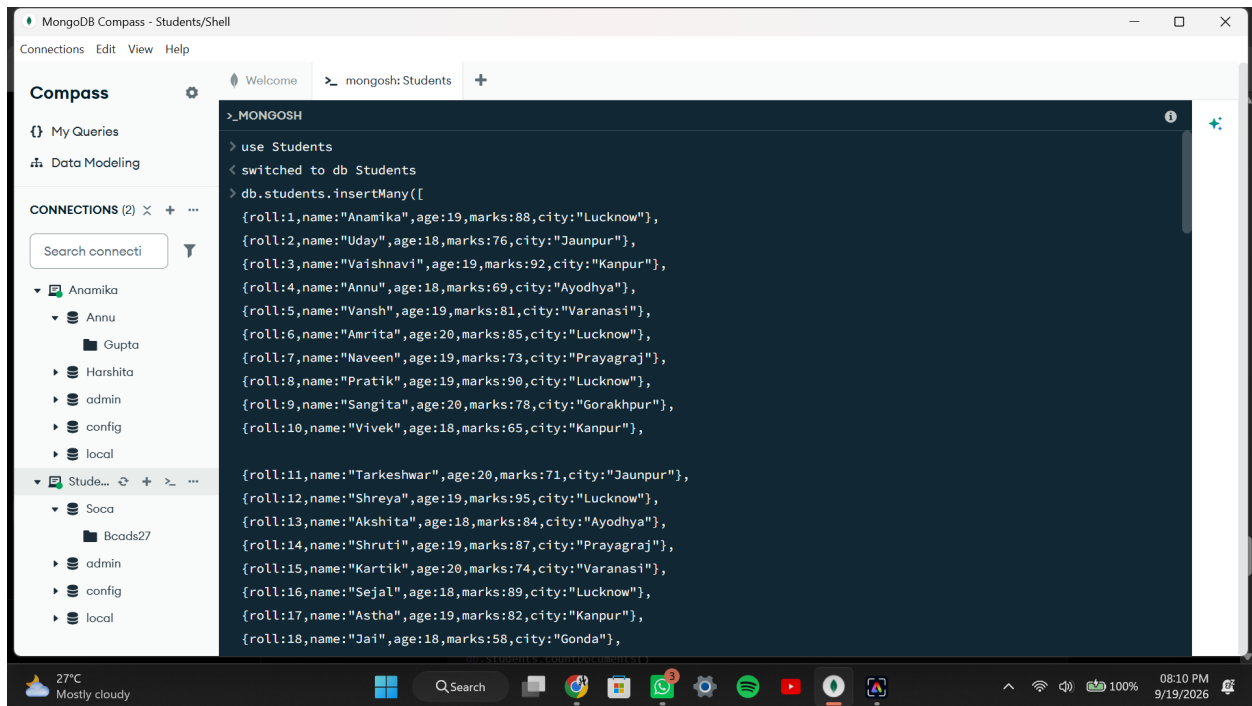
Roll	Name	Age	Marks	City
41	Meenu	18	74	Jaunpur
42	Akanksha	20	89	Lucknow
43	Sarika	19	62	Ayodhya
44	Anchal	18	78	Prayagraj
45	Umra	20	84	Lucknow
46	Nashra	19	73	Kanpur
47	Aman	18	59	Gonda
48	Priya	20	87	Lucknow
49	Rahul	19	64	Bareilly
50	Neha	18	91	Varanasi

5. MONGODB IMPLEMENTATION AND OPERATIONS

The following sections document the operations performed in the project. The supplied reference document presents the MongoDB commands through terminal/console screenshots; those screenshots are retained here instead of placing the command text directly in the report.

1. Create Database

The project starts by creating or switching to the Students database. This provides the database context in which the student collection is maintained.



```
> use Students
switched to db Students
> db.students.insertMany([
  {roll:1,name:"Anamika",age:19,marks:88,city:"Lucknow"},
  {roll:2,name:"Uday",age:18,marks:76,city:"Jaunpur"},
  {roll:3,name:"Vaishnavi",age:19,marks:92,city:"Kanpur"},
  {roll:4,name:"Annu",age:18,marks:69,city:"Ayodhya"},
  {roll:5,name:"Vansh",age:19,marks:81,city:"Varanasi"},
  {roll:6,name:"Amrita",age:20,marks:85,city:"Lucknow"},
  {roll:7,name:"Naveen",age:19,marks:73,city:"Prayagraj"},
  {roll:8,name:"Pratik",age:19,marks:90,city:"Lucknow"},
  {roll:9,name:"Sangita",age:20,marks:78,city:"Gorakhpur"},
  {roll:10,name:"Vivek",age:18,marks:65,city:"Kanpur"},
  {roll:11,name:"Tarkeshwar",age:20,marks:71,city:"Jaunpur"},
  {roll:12,name:"Shreya",age:19,marks:95,city:"Lucknow"},
  {roll:13,name:"Akshita",age:18,marks:84,city:"Ayodhya"},
  {roll:14,name:"Shruti",age:19,marks:87,city:"Prayagraj"},
  {roll:15,name:"Kartik",age:20,marks:74,city:"Varanasi"},
  {roll:16,name:"Sejal",age:18,marks:89,city:"Lucknow"},
  {roll:17,name:"Astha",age:19,marks:82,city:"Kanpur"},
  {roll:18,name:"Jai",age:18,marks:58,city:"Gonda"}
])
```

Figure: MongoDB execution screenshot — Create Database

2. Create Collection

A collection named students is used to store the student documents. MongoDB collections serve as the document-oriented equivalent of a table in this project.

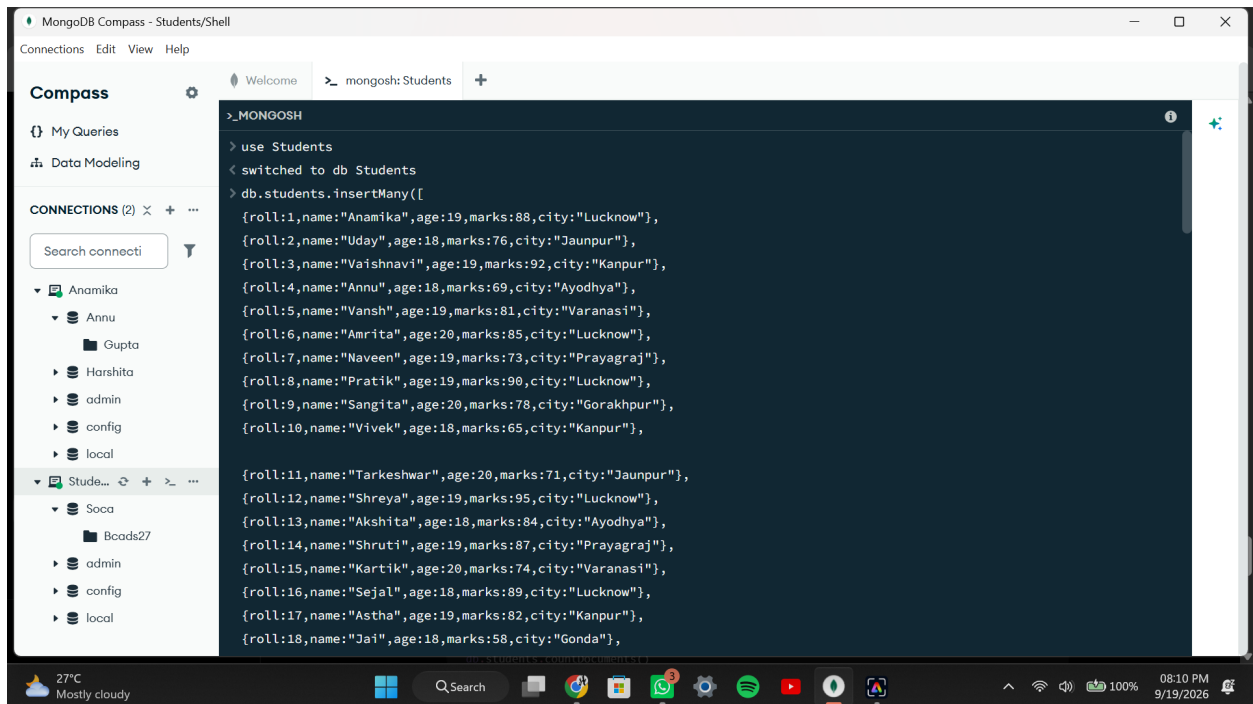


Figure: MongoDB execution screenshot — Create Collection

3. Insert 50 Students

Multiple student documents are inserted into the students collection. The dataset contains 50 records with roll number, name, age, marks and city fields.

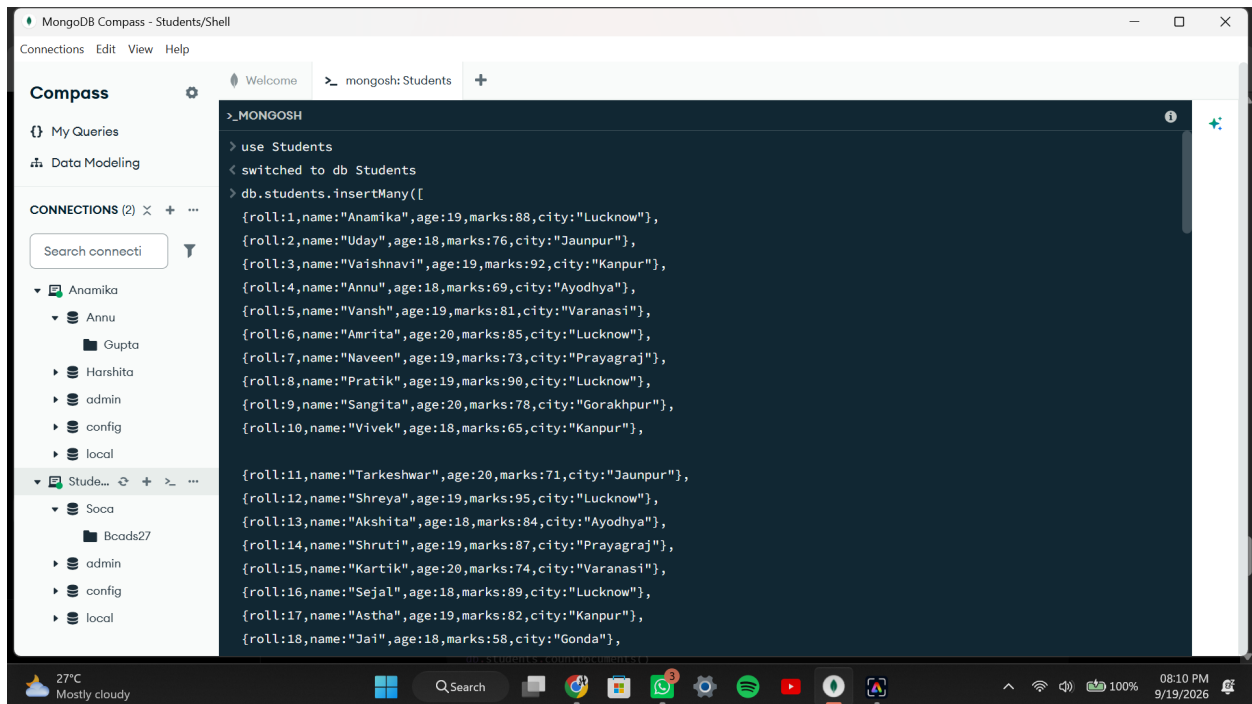


Figure: MongoDB execution screenshot — Insert 50 Students

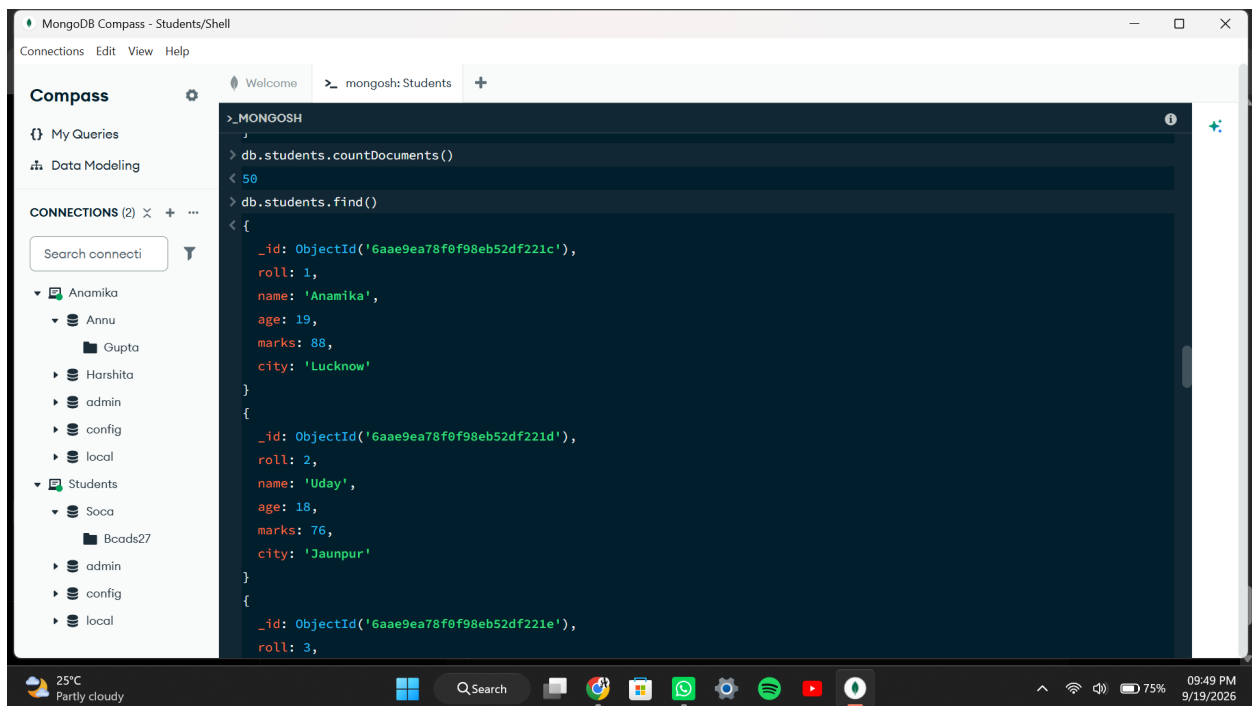


Figure: MongoDB execution screenshot — Insert 50 Students (continued)

4. Display All Students

The collection is queried to retrieve student documents. The execution output demonstrates that the inserted student information can be displayed from MongoDB.

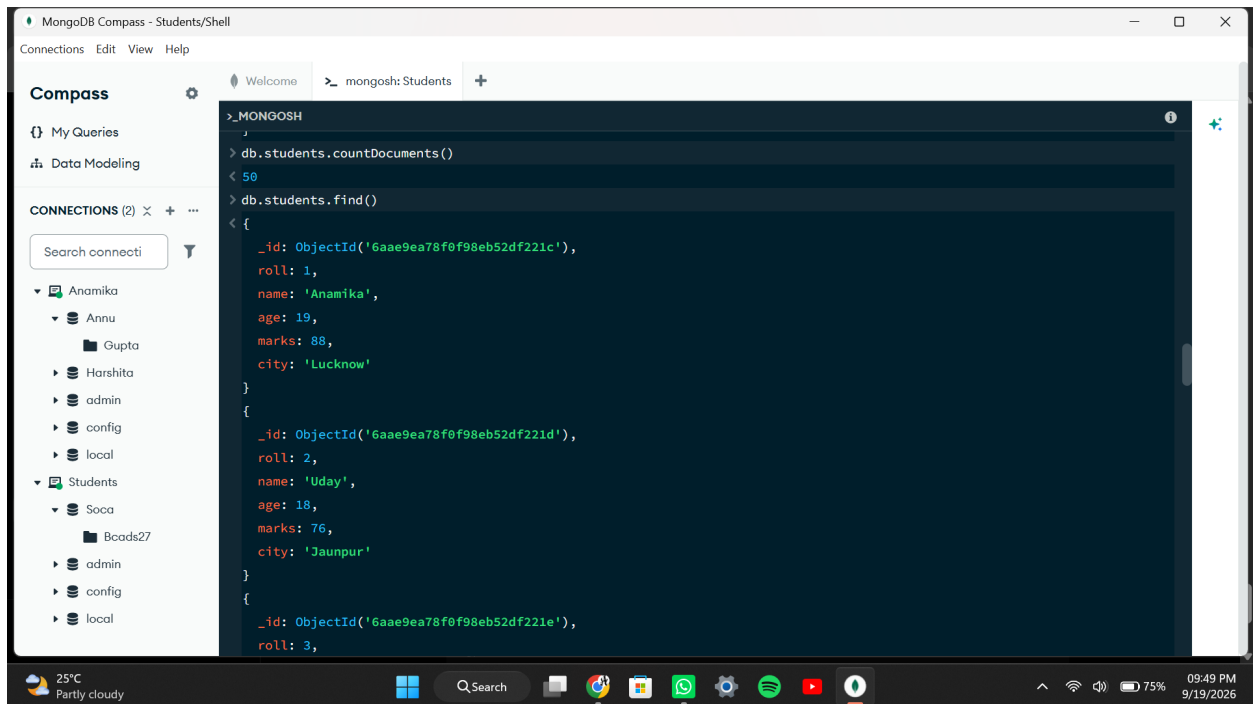


Figure: MongoDB execution screenshot — Display All Students

5. Equal To — \$eq

The equality comparison is used to find students whose marks are exactly 88.

```
> db.students.find({marks: {$eq: 88}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2236'),
  roll: 27,
  name: 'Ananya',
  age: 19,
  marks: 88,
  city: 'Varanasi'
}
```

Figure: MongoDB execution screenshot — Equal To — \$eq

6. Greater Than — \$gt

The greater-than comparison is used to find students whose marks are greater than 85.

```
> db.students.find({marks: {$gt: 85}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2223'),
  roll: 8,
  name: 'Pratik',
  age: 19,
  marks: 90,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2227'),
  roll: 12,
  name: 'Shreya',
  age: 19,
  marks: 95,
  city: 'Lucknow'
}
```

Figure: MongoDB execution screenshot — Greater Than — \$gt

7. Less Than — \$lt

The less-than comparison is used to find students whose marks are below 60.

```
> db.students.find({marks: {$lt: 60}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df222d'),
  roll: 18,
  name: 'Jai',
  age: 18,
  marks: 58,
  city: 'Gonda'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df223c'),
  roll: 33,
  name: 'Jai',
  age: 20,
  marks: 55,
  city: 'Ayodhya'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df224a'),
  roll: 47,
  name: 'Aman',
  age: 18,
  marks: 59,
  city: 'Gonda'
}
> db.students.find({marks: {$eq: 88}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
```

Figure: MongoDB execution screenshot — Less Than — \$lt

8. Greater Than or Equal To — \$gte

The greater-than-or-equal comparison is used to find students whose marks are 90 or above.

```
> db.students.find({marks: {$gte: 90}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2223'),
  roll: 8,
  name: 'Pratik',
  age: 19,
  marks: 90,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2227'),
  roll: 12,
  name: 'Shreya',
  age: 19,
  marks: 95,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df222e'),
  roll: 19,
  name: 'Harshita',
  age: 20,
  marks: 91,
  city: 'Lucknow'
}
```

Figure: MongoDB execution screenshot — Greater Than or Equal To — \$gte

9. Less Than or Equal To — \$lte

The less-than-or-equal comparison is used to find students whose marks are 70 or below.

```
> db.students.find({marks: {$lte: 70}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221f'),
  roll: 4,
  name: 'Annu',
  age: 18,
  marks: 69,
  city: 'Ayodhya'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2225'),
  roll: 10,
  name: 'Vivek',
  age: 18,
  marks: 65,
  city: 'Kanpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df222d'),
  roll: 18,
  name: 'Jai',
  age: 18,
  marks: 58,
  city: 'Gonda'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df222f'),
  roll: 20,
  name: 'Omkar',
  age: 19,
  marks: 63,
  city: 'Bareilly'
}
```

Figure: MongoDB execution screenshot — Less Than or Equal To — \$lte

10. Update Operation

An existing student document is updated by changing the marks of the student with roll number 5 to 95.

```
> _MONGOSH
}
> db.students.updateOne(
  {roll:5},
  {$set:{marks:95}}
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.students.find({roll:5})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df2220'),
  roll: 5,
  name: 'Vansh',
  age: 19,
  marks: 95,
  city: 'Varanasi'
}
```

Figure: MongoDB execution screenshot — Update Operation

11. Delete One Operation

A single document matching roll number 10 is deleted from the collection.

```
}  
> db.students.deleteOne({roll:10})  
< {  
  acknowledged: true,  
  deletedCount: 1  
}  
> db.students.find({roll:10})  
<  
Students >
```

Figure: MongoDB execution screenshot — Delete One Operation

12. Delete Many Operation

All student documents having marks below 60 are removed using a multiple-document deletion operation.

```
> db.students.deleteMany({marks:{$lt:60}})
< {
  acknowledged: true,
  deletedCount: 3
}
> db.students.find({marks:{$lt:60}})
<
Students > |
```

Figure: MongoDB execution screenshot — Delete Many Operation

13. AND Operation — \$and

The logical AND operation finds students who satisfy both conditions: marks greater than 80 and age equal to 19.

```
> _MONGOSH
> db.students.find({
  $and: [
    {marks: {$gt: 80}}, {age: 19}
  ]
})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
```

Figure: MongoDB execution screenshot — AND Operation — \$and

14. OR Operation — \$or

The logical OR operation finds students whose city is Lucknow or Kanpur.

```
>_MONGOSH
> db.students.find({
  $or:[
    {city:"Lucknow"},{city:"Kanpur"}
  ]
})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
```

Figure: MongoDB execution screenshot — OR Operation — \$or

15A. Sort Operation — Ascending

Student records are sorted by marks from lowest to highest. Ascending sorting uses the positive sort direction shown in the project.

```
> _MONGOSH
> db.students.find().sort({marks:1})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df2237'),
  roll: 28,
  name: 'Bhaskar',
  age: 18,
  marks: 61,
  city: 'Gorakhpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2246'),
  roll: 43,
  name: 'Sarika',
  age: 19,
  marks: 62,
  city: 'Ayodhya'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df222f'),
  roll: 20,
  name: 'Omkar',
  age: 19,
  marks: 63,
  city: 'Bareilly'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df224c'),
  roll: 49,
  name: 'Rahul',
  age: 19,
  marks: 64,
  city: 'Bareilly'
}
```

Figure: MongoDB execution screenshot — Sort Operation — Ascending

15B. Sort Operation — Descending

Student records are sorted by marks from highest to lowest. Descending sorting uses the negative sort direction shown in the project.

```
>_MONGOSH
> db.students.find().sort({marks:-1})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df223e'),
  roll: 35,
  name: 'Krishna',
  age: 19,
  marks: 96,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2227'),
  roll: 12,
  name: 'Shreya',
  age: 19,
  marks: 95,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2220'),
  roll: 5,
  name: 'Vansh',
  age: 19,
  marks: 95,
  city: 'Varanasi'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2235'),
  roll: 26,
  name: 'Anushka',
  age: 20,
  marks: 94,
  city: 'Lucknow'
}
```

Figure: MongoDB execution screenshot — Sort Operation — Descending

16. Limit Operation

The limit operation restricts the displayed result to the first five documents.

```
>_MONGOSH
> db.students.find().limit(5)
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221d'),
  roll: 2,
  name: 'Uday',
  age: 18,
  marks: 76,
  city: 'Jaunpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
```

Figure: MongoDB execution screenshot — Limit Operation

17. Search by City

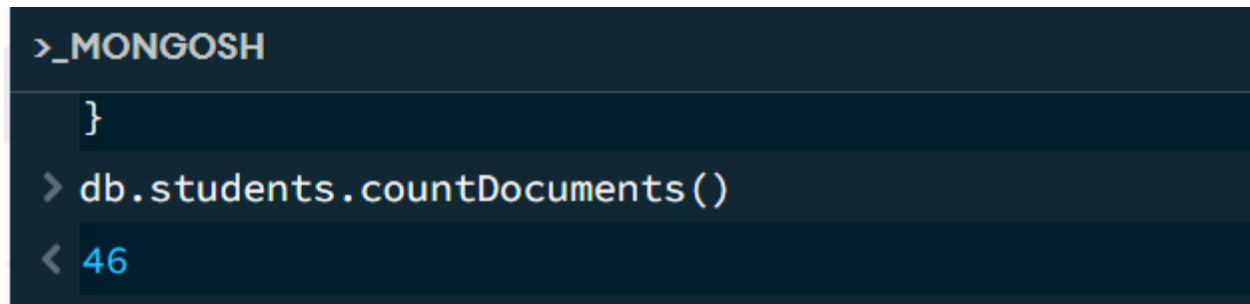
The collection is filtered to find all student documents whose city is Lucknow.

```
>_MONGOSH
> db.students.find({city:"Lucknow"})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2221'),
  roll: 6,
  name: 'Amrita',
  age: 20,
  marks: 85,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2223'),
  roll: 8,
  name: 'Pratik',
  age: 19,
```

Figure: MongoDB execution screenshot — Search by City

18. Count Documents

The count operation reports the number of documents currently present after the deletion operations. The reference screenshot shows a count of 46.



```
>_MONGOSH
}
> db.students.countDocuments()
< 46
```

Figure: MongoDB execution screenshot — Count Documents

6. MONGODB OPERATORS USED

The project demonstrates the following MongoDB comparison and logical operators:

Operator	Meaning in this project	Demonstrated use
\$eq	Equal to	Marks exactly 88
\$gt	Greater than	Marks greater than 85
\$lt	Less than	Marks below 60
\$gte	Greater than or equal to	Marks 90 or above
\$lte	Less than or equal to	Marks 70 or below
\$and	All specified conditions must be satisfied	Marks > 80 and age = 19
\$or	At least one specified condition is satisfied	City = Lucknow or Kanpur

CRUD operations are also represented through the project workflow: student records are created/inserted, retrieved/displayed, updated and deleted. Sorting, filtering, limiting and counting are included as additional data-management operations.

7. CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update and delete documents. I also learned how ascending and descending order work, how comparison operators such as \$eq, \$gt, \$lt, \$gte and \$lte are used, and how logical operators such as \$and and \$or work.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.

PROJECT DETAILS

Student Name	Diwakar Mishra
Roll No.	33
Class	BCA (DS & AI) 23
Teacher	Mr. Harendra Singh Rajpoot
Project	Student Management System Using MongoDB
Database	Students
Collection	students
Initial Records	50
Count shown after deletions	46

University Roll No. 1250258198

8. APPENDIX — COMPLETE STUDENT DATASET

For completeness, the 50 student documents from the supplied reference project are reproduced below as a readable data table rather than as direct MongoDB code.

Roll	Name	Age	Marks	City
1	Anamika	19	88	Lucknow
2	Uday	18	76	Jaunpur
3	Vaishnavi	19	92	Kanpur
4	Annu	18	69	Ayodhya
5	Vansh	19	81	Varanasi
6	Amrita	20	85	Lucknow
7	Naveen	19	73	Prayagraj
8	Pratik	19	90	Lucknow
9	Sangita	20	78	Gorakhpur
10	Vivek	18	65	Kanpur
11	Tarkeshwar	20	71	Jaunpur
12	Shreya	19	95	Lucknow
13	Akshita	18	84	Ayodhya
14	Shruti	19	87	Prayagraj
15	Kartik	20	74	Varanasi
16	Sejal	18	89	Lucknow
17	Astha	19	82	Kanpur
18	Jai	18	58	Gonda
19	Harshita	20	91	Lucknow
20	Omkar	19	63	Bareilly
21	Ayush	18	77	Lucknow
22	Saumya	19	86	Jaunpur
23	Vidya	20	72	Kanpur
24	Adarsh	19	80	Ayodhya
25	Anmol	18	67	Prayagraj

APPENDIX — COMPLETE STUDENT DATASET (CONTINUED)

Roll	Name	Age	Marks	City
26	Anushka	20	94	Lucknow
27	Ananya	19	88	Varanasi
28	Bhaskar	18	61	Gorakhpur
29	Rohan	19	75	Kanpur
30	Nishant	20	79	Lucknow
31	Vineet	18	68	Jaunpur
32	Anika	19	90	Lucknow
33	Jai	20	55	Ayodhya
34	Lakshya	18	83	Kanpur
35	Krishna	19	96	Prayagraj
36	Rumana	20	70	Lucknow
37	Priyanshu	18	81	Varanasi
38	Aditya	19	66	Gorakhpur
39	Dhruv	20	93	Lucknow
40	Nivedita	19	85	Kanpur
41	Meenu	18	74	Jaunpur
42	Akanksha	20	89	Lucknow
43	Sarika	19	62	Ayodhya
44	Anchal	18	78	Prayagraj
45	Umra	20	84	Lucknow
46	Nashra	19	73	Kanpur
47	Aman	18	59	Gonda
48	Priya	20	87	Lucknow
49	Rahul	19	64	Bareilly
50	Neha	18	91	Varanasi

End of Project Report